DESIGN REVIEW · SAR COVERAGE PLANNER

Flight Planner Design Review

A designer's read of the multi-drone search-and-rescue mission planner: how it is put together, why it was built this way, what ten SITL sorties actually showed, and the places where the design and its capabilities run out.

Reviewed **22 Sep 2026** Source **2,726 LOC** / 13 modules Tests **135 passing**
Sorties **10 SITL, 8 tabulated** Field trials **none**

01

The problem it solves

Turn a hand-drawn operational area into flyable missions for several drones at once, and refuse to emit anything that would fly somewhere it must not.

The input is a KML with three things in it: a launch point, a **GeoFence** that is a hard boundary, and an **OpsArea** that is the ground to be searched. The output is one QGroundControl `.plan` per drone — a complete flight, takeoff through return — plus a combined plan carrying all of them.

The governing design decision is that **altitude is the only imaging knob**. Pick a height and the camera geometry of the SIYI ZR30 re-derives the ground swath, the lane spacing, the ground sample distance, and the blur-safe cruise speed. Nothing downstream is allowed to invent a spacing or a speed by hand. At 30 m that chain produces a 33.3 m swath, 2.60 cm/px, 25.0 m lanes and 13.0 m/s — and if any one of those is edited independently, the others are quietly wrong.

Everything else — how many drones, which partition strategy, how corners are shaped, whether the swarm flies in as a formation — is configuration layered on top of that single derivation.

02

Architecture

Thirteen modules, no runtime dependencies in the core package. The dependency graph is acyclic and deliberately shallow: geometry knows nothing about missions, the plan writer knows nothing about how waypoints were chosen, and the safety check knows nothing about how to make a plan — only how to reject one.

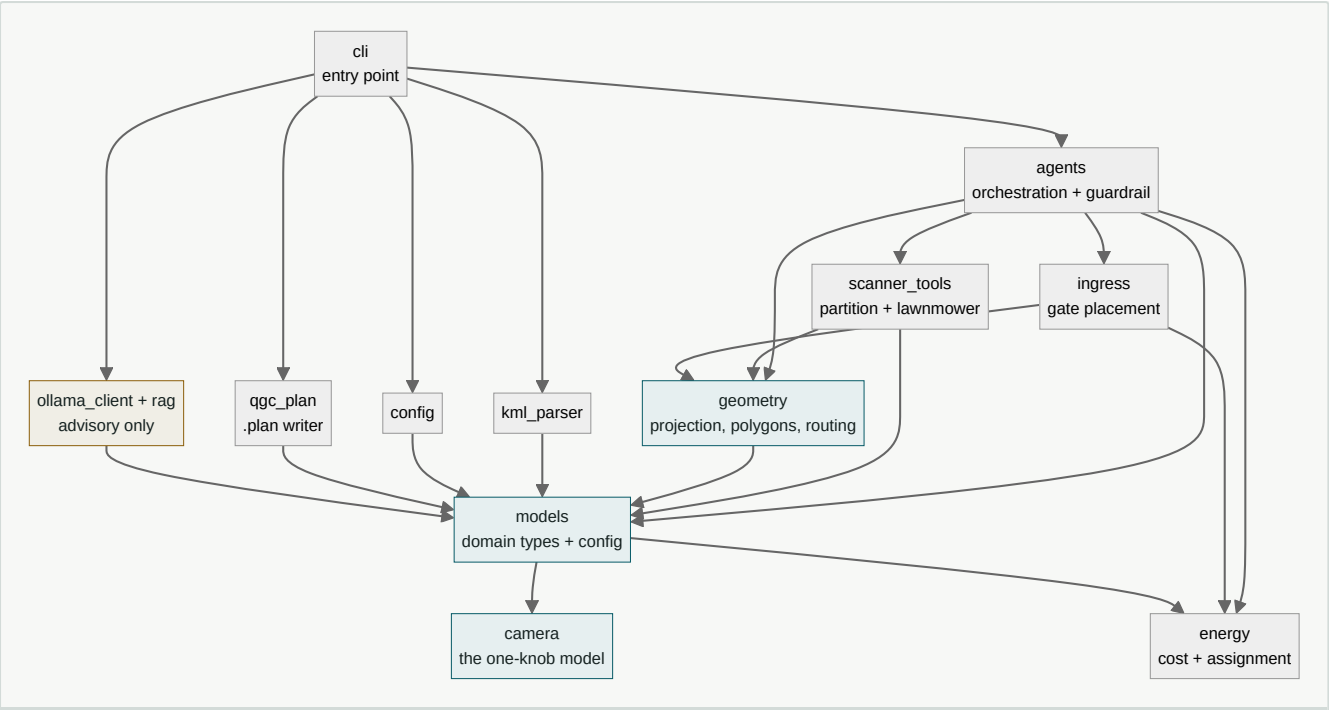


FIG 1 Module dependencies. `models` and `camera` sit at the bottom as the shared vocabulary; the LLM briefing hangs off the side, where a failure cannot affect the mission.

The domain model is small and almost entirely immutable. `PlannerConfig` is a frozen dataclass that validates itself, and every derived imaging number is a property computed from altitude rather than a stored field — which is what makes the single-knob rule structural instead of a convention people have to remember.

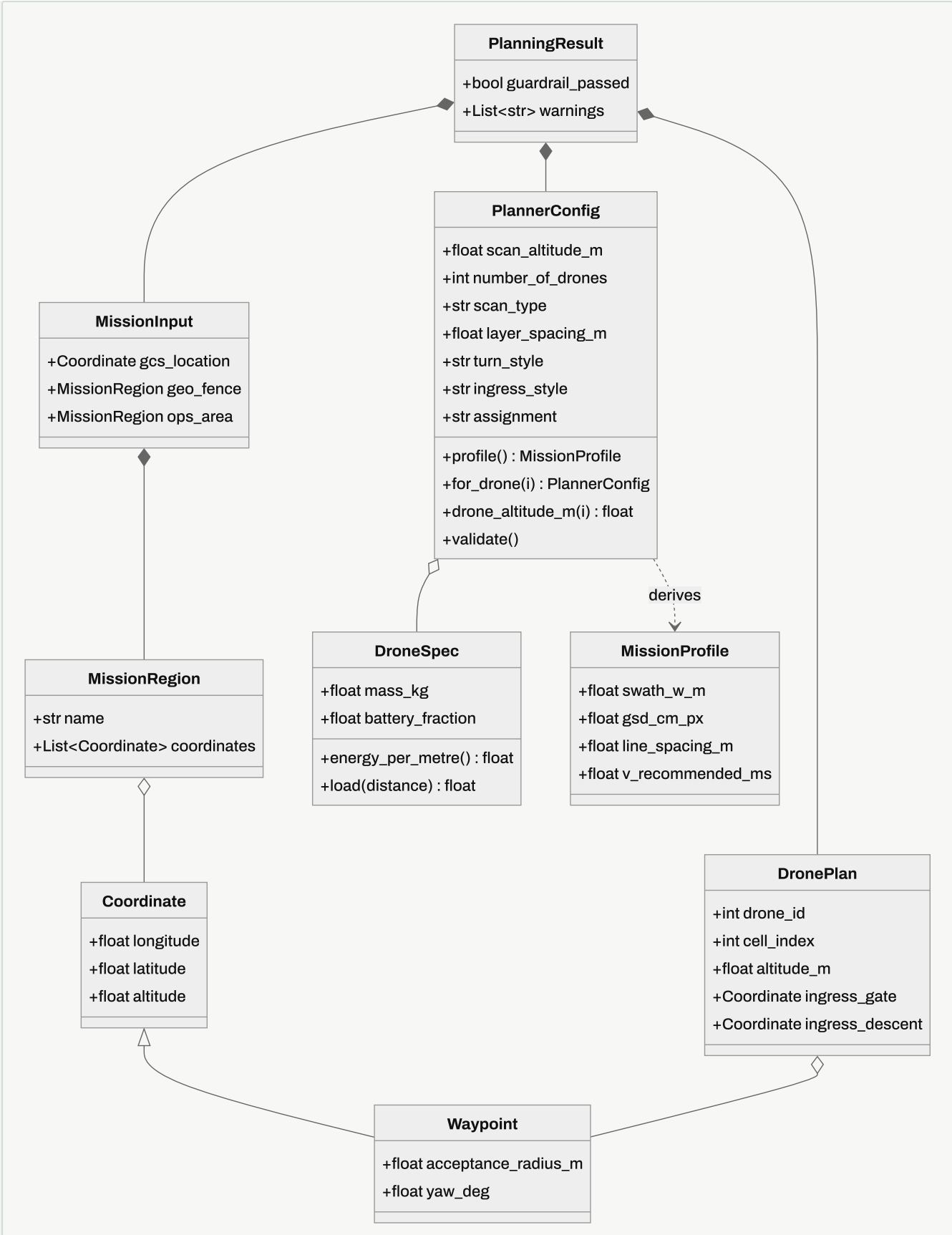


FIG 2 Domain model. `MissionProfile` is derived, never stored — the arrow is a dependency, not composition.

A planning run

One call produces every drone's mission. The order matters: the area is cut before anyone knows who is flying it, work is handed out before the routes are laid, and the safety check runs last on the finished article rather than on intentions.

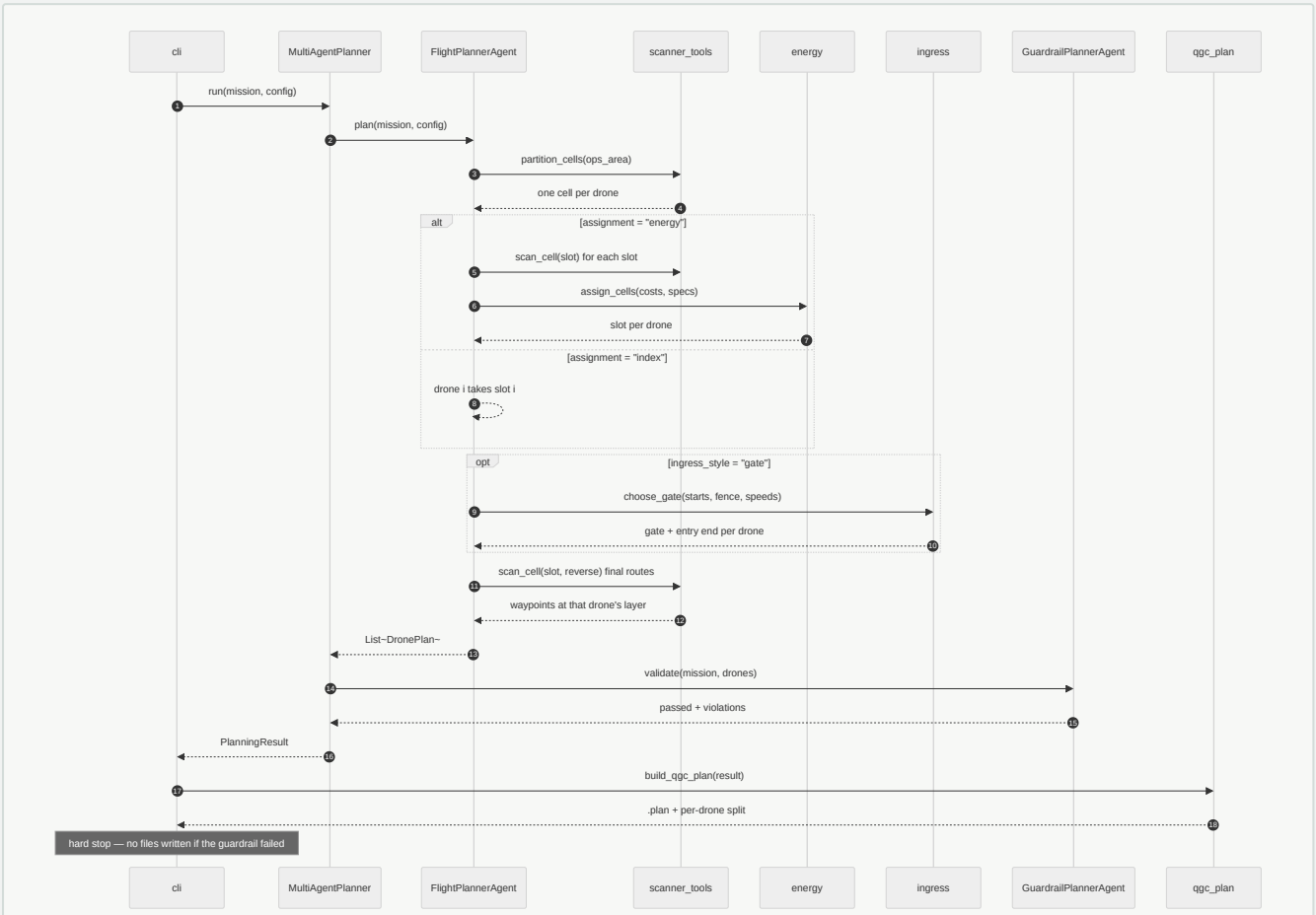
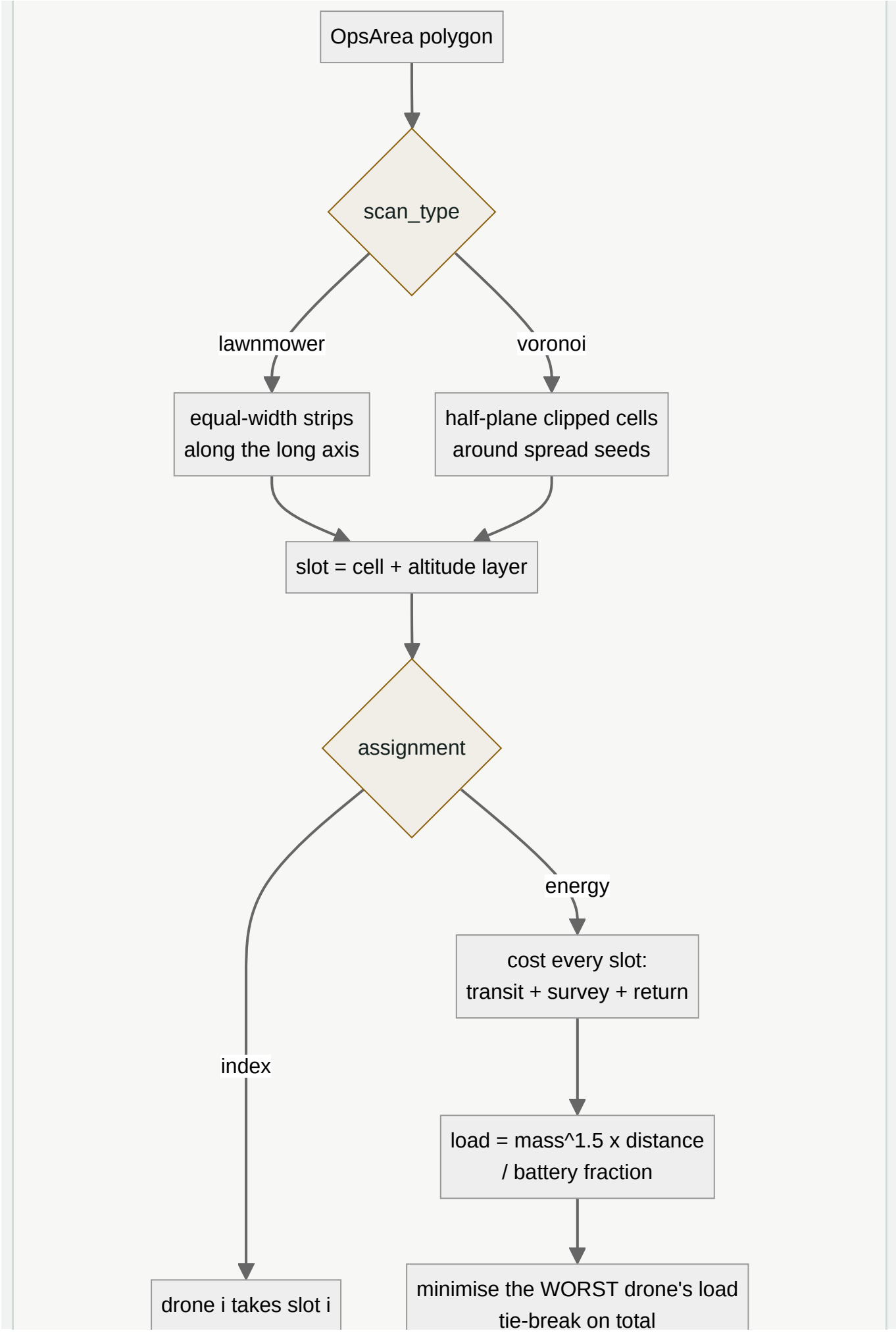


FIG 3 A planning run. The guardrail is a separate participant on purpose: it can only reject.

Two decisions inside that run carry most of the design thinking. The first is what a unit of work actually is. A **slot** is a cell *and* the altitude layer it is flown at, because the layer sets lane spacing and therefore the length of the survey — the lowest layer flies the tightest lanes and the longest route. Treating the cell alone as the unit was the original design, and it measured a 0.0 % improvement everywhere: the heavy, half-charged drone stayed stranded on the most expensive layer and no reshuffle of cells could rescue it.


OpsArea polygon
scan_type
lawnmower
voronoi
equal-width strips
along the long axis
half-plane clipped cells
around spread seeds
slot = cell + altitude layer
assignment
energy
cost every slot:
transit + survey + return
index
load = mass^1.5 x distance
/ battery fraction
drone i takes slot i
minimise the WORST drone's load
tie-break on total

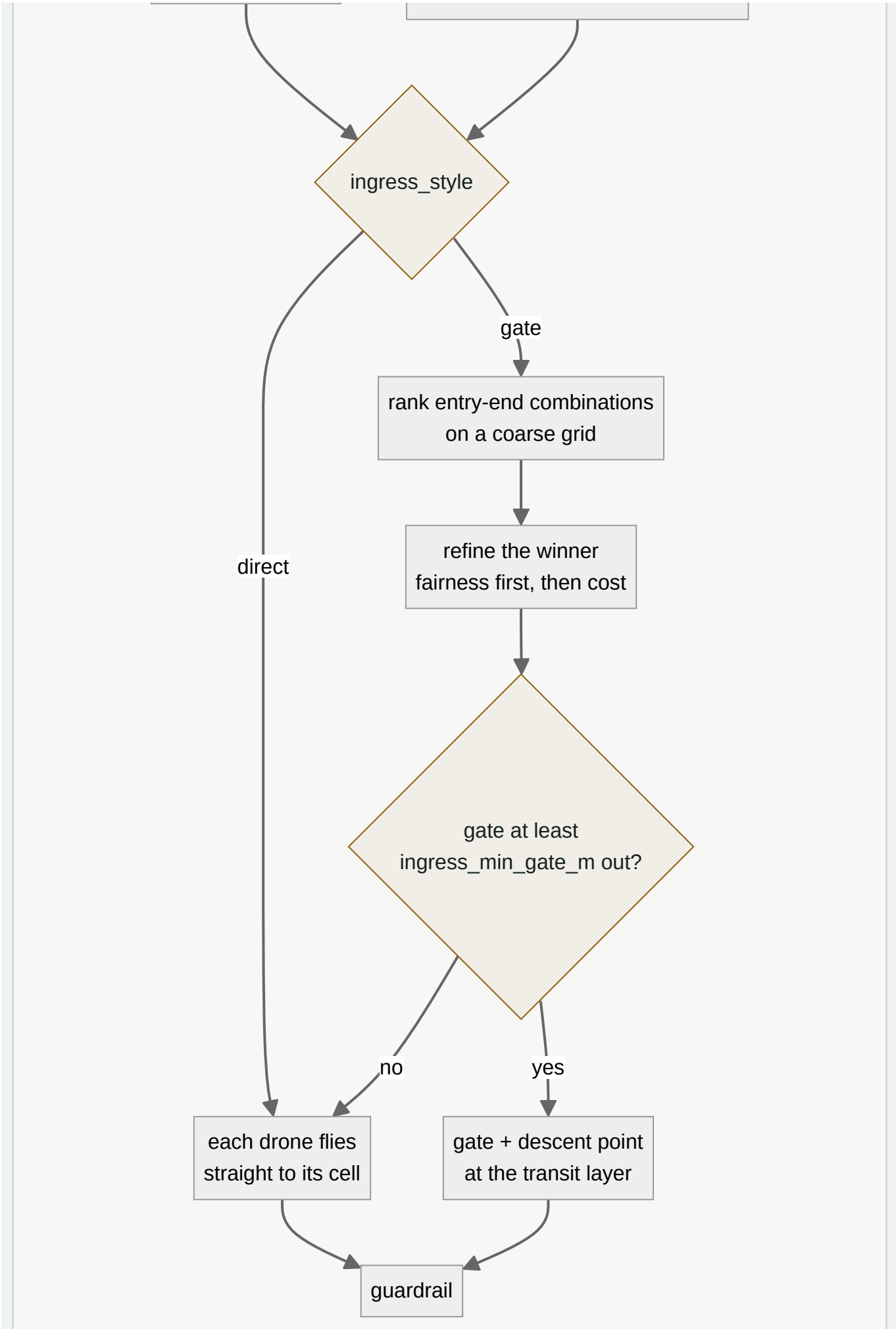

ingress_style
direct
gate
rank entry-end combinations
on a coarse grid
refine the winner
fairness first, then cost
gate at least
ingress_min_gate_m out?
no
yes
each drone flies
straight to its cell
gate + descent point
at the transit layer
guardrail

FIG 4 How work is divided and where the swarm splits up. The two diamonds on the right are the 16 Sep addition.

The second is the shape of a flight once a gate exists. A drone holds its transit layer past the gate and only drops to its survey layer part-way down the fan-out leg — descending at the gate itself would stack every aircraft's altitude change in the one place they are all on top of each other.

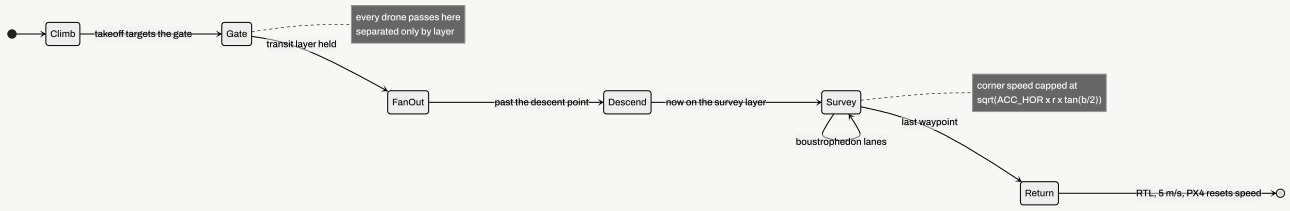


FIG 5 Mission phases with a dispersal gate. Each phase is flown at a known altitude, which is what lets separation be judged leg by leg.

04

Methodology & rationale

Derive, never duplicate

Every imaging number is a property of altitude. The alternative — storing spacing and speed as configuration — was tried in the inherited codebase and produced a plan asking for 15 m/s cruise with no link to the optics at all. Deriving them means a low-altitude scan cannot silently motion-blur every frame.

Planning and safety are different agents

The guardrail projects the original KML fence, then checks every waypoint, every leg, every rounded corner as PX4 would actually fly it, and both transits the planner never chose — home to the first waypoint, and the return. It has no ability to fix anything. That separation is why a bug in lane generation shows up as a refusal rather than a plausible-looking illegal mission, and it caught a real one: 966 m of a Toa Payoh survey crossing an excluded corridor on eight legs.

Fairness objectives are lexicographic, with a tolerance

Placing the dispersal gate is an optimisation with a degenerate solution set — the points where three drones arrive simultaneously form a curve, not a point, and most of that curve is absurd. The first implementation happily returned a gate 708 m out costing 1,417 m of ingress each, because it was fair to nine decimal places where a sensible gate was fair to four. The objective now returns a

pair, fairness first and cost second, with fairness bucketed to a tolerance — 0.5 s, because precision finer than wind and the corner model is not real.

Regression by byte-identity

Every new capability defaults to the previous behaviour, and a harness re-plans three sites × two scan types × five fleet sizes and compares the output against the previous code. The ingress work landed with all 30 plans item-for-item identical. This is the single most useful discipline in the project: it makes "nothing changed unless you asked" a checkable claim rather than a hope.

The reference file is tested against the code

`parameters.yaml` documents every knob and a test asserts its values match the live source, so the documentation cannot silently drift. Its limits are discussed in §7.

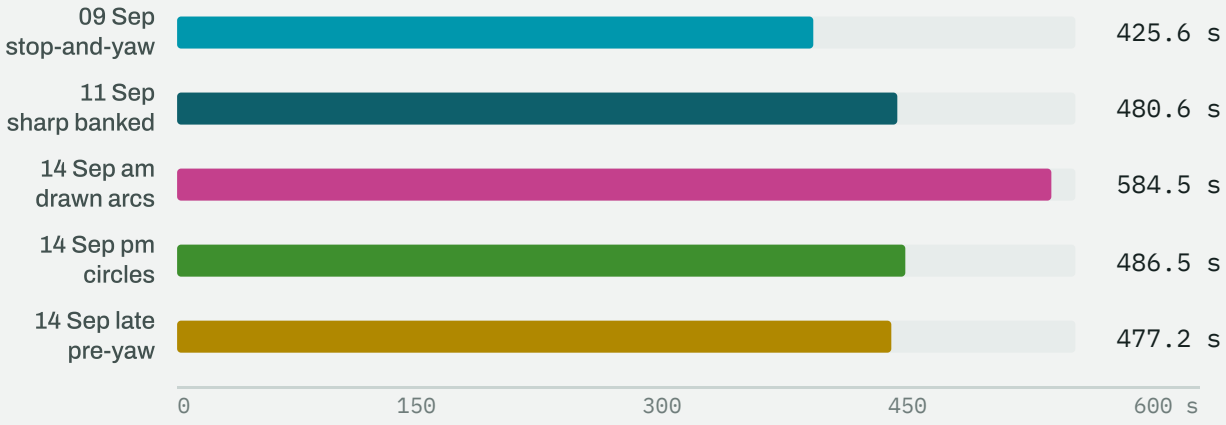
05

What the flights showed

Ten sorties, all in PX4 SITL with Gazebo; eight are in the results table and the two most recent tuning runs are not yet tabulated. Tracking accuracy has been consistent throughout and is not the interesting result — cross-track mean has sat between 0.13 m and 0.46 m since the first flight. What the flights actually settled were questions about *speed* and *bank*.

The turn series: four drawings, one wall

Four successive attempts to make the aircraft hold a smooth, aircraft-like bank through a lane-end turn, all flown on the same Toa Payoh sector with the same drone.



Survey time, same sector and drone. The 09 Sep run flew a different (invalid) plan with 14 corners instead of 36.

TYPICAL TURN, PEAK BANK AGAINST THE 15° TARGET

VERSION	PEAK BANK	TURNS PAST 15°	CROSS-TRACK MEAN
Stop-and-yaw	6.7°	0 / 15	0.23 m
Sharp banked	17.3°	36 / 37	0.24 m
Drawn arcs	14.1°	2 / 21	0.13 m
Circles	16.2°	20 / 21	0.22 m
Pre-yaw	10.9°	16 / 37	0.24 m

> **No waypoint geometry produces a steady bank** ARCHITECTURAL
>
> PX4 re-plans a straight line to each new waypoint on every switch and uses the acceptance radius only to cap corner speed. Four different drawings hit the same wall: the bank arrives as one pulse per vertex, not one held angle. A genuinely steady bank needs a continuous trajectory streamed from the companion computer, not a cleverer `.plan`.

The pre-yaw attempt is worth recording as a negative result with a clean explanation. Turning the nose onto the next leg before the corner does exactly what it claims — and changes nothing about the bank, because a multirotor turns by tilting toward the new direction, not by yawing. Total tilt matched the sharp corners (95th percentile 19.5° against 19.3°) while peak roll became the worst of any version at 22.2°: the braking simply moved out of pitch and into roll.

The real speed limit is geometry, not tuning

Measured 16 Sep: a survey commanded at 12.2 m/s averaged **8.13 m/s** and spent 40 % of its time below 6 m/s — 12 m/s down each lane, braking to ~3.5 m/s at every turn, 13 slow-downs in a 300 s window. Altitude held to 0.25 m throughout, so this was never instability. PX4's per-waypoint speed cap is `sqrt(MPC_ACC_HOR × param2 × tan(bend/2))`, which on a drawn-circle vertex works out at 2.28 m/s. Holding the commanded cruise around the automatic 11.65 m turn radius would need 52.5° of bank against a 25° ceiling.

> **Altitude silently caps turn speed far below the cruise it produces** CONSEQUENCE
>
> Turn radius auto-derives as half the lane spacing, and lane spacing comes from altitude. So the one knob sets a blur-safe cruise *and*, independently, a corner speed that the cruise cannot reach. Every mission-duration and endurance figure derived from cruise speed is roughly 50 % optimistic.

Tuning the acceleration, and overshooting

Raising `MPC_ACC_HOR` raises both the bank angle and the corner-speed cap. Flown on the same Toa Payoh plan, 18 Sep, SITL:

MPC_ACC_HOR TRADE-OFF

SETTING	IMPLIED BANK	SURVEY MEDIAN	PEAK BANK	CROSS-TRACK MEAN
2.6 m/s² `(11 Sep)`	14.8°	459.7 s	15.68°	0.22 m
4.5 m/s² `(18 Sep, flown)`	24.6°	407.5 s	24.54°	0.74 m
3.5 m/s² `(22 Sep, set)`	19.6°	not flown	not flown	not flown

4.5 m/s² bought 11.3 % off the survey and cut time below 4 m/s by 71 %. It also ran the aircraft into its own tilt ceiling: median peak bank 24.54° against `MPC_TILTMAX_AIR` 25°, on essentially every turn. The diagnostic detail is that at 4.5 the aircraft stopped *overshooting* its commanded bank the way it does at 2.6 — the limiter was clamping it. Where a turn needed more than 25° the controller could not deliver it and ran wide, and cross-track mean more than tripled. 3.5 m/s² is the correction, leaving 5.4° of margin; **it has not yet been flown.**

Group ingress and equitable dispersal

Added 16 Sep. Two findings sit underneath it, both geometric rather than empirical.

- **A gate can never save energy.** Home → gate → cell start is never shorter than home → cell start, so grouping up costs every drone distance. Asked to minimise fleet energy, the solver correctly placed the gate at home — that is, "do not group up".
- **Matching work to aircraft is free.** A permutation cannot change the total distance flown, only who pays it. On an illustrative heterogeneous fleet this took 16.9 % off the worst drone's battery draw at Toa Payoh and 31.7 % at Bedok, with fleet distance identical to the metre — and 0.0 % on three other sector/pattern combinations where the binding drone could not be helped.

The gate's real product is a simultaneous start: drones that began searching 39 s apart now begin within 0.2 s, for roughly 3–7 % more total battery, since ingress is only 4–14 % of a flight.

Scale, and where it breaks

BEDOK RESERVOIR, 82 HA OF WATER, 3 VS 8 DRONES

MEASURE	3 DRONES	8 DRONES
Longest sortie	14.0–21.0 min	4.3–13.0 min
Arrival spread with a gate	0.2 s	30–34 s
Workload spread, lawnmower	1.58×	3.15×
Workload spread, voronoi	1.24×	2.11×
Layers used	28–32 m	24–38 m

Eight drones solve the endurance problem and create three new ones: the gate stops being able to equalise arrivals (eight constraints, a point with two degrees of freedom), the top two layers pass the 3 cm/px resolution standard, and the transit and survey altitude ladders interleave. Those are covered in §7.

06

What the design gets right

> **One derivation, no duplicated constants** STRUCTURAL
>
> Spacing, GSD and cruise speed are properties computed from altitude, not fields. It is not possible for them to disagree, because there is only one of each.

> **Safety is a separate agent that can only refuse** STRUCTURAL
>
> The guardrail checks waypoints, legs, PX4's actual rounded corners, both unchosen transits, and inter-drone separation, and the CLI writes nothing on failure. Real breaches have been caught this way rather than flown.

> **Behaviour changes are opt-in and provable** PROCESS
>
> Defaults preserve previous behaviour and a 30-plan harness proves it item-for-item. The most recent feature set — energy-aware assignment, the dispersal gate and three new self-checks — landed with all 30 comparison plans identical to the code before it.

> **Deterministic and dependency-free** STRUCTURAL
>
> No randomness on the planning path and no third-party packages in the core, so a plan is reproducible from its inputs and the package will still import in five years. A test asserts two identical runs produce identical plans.

> **Negative results are recorded as carefully as positive ones** PROCESS
>
> The pre-yaw attempt, the 4.5 m/s^2 overshoot and the 0.0 % assignment cases are all written down with their numbers. This is why the turn series reads as one argument instead of four unrelated experiments.

07

Where it is flawed

The energy model is a guess wearing a number CRITICAL

Energy per metre is modelled as mass$^{1.5}$ with no absolute capacity anywhere — the units are relative. It can rank drones against each other, which is enough to assign work, but it *cannot answer whether a mission is flyable on a battery*. A 20-minute sortie that exceeds endurance will plan, validate and emit without complaint. It needs hover current against all-up mass and cruise power against speed, measured on the airframe.

Two altitude ladders that interleave CRITICAL

Survey layers step by 2 m and transit layers by 5 m from the same base, so for a large enough fleet a drone transits at 25 m while another surveys at 26 m — and the ingress crosses that drone's cell. This is now refused at configuration time with both layers named, but the underlying design has no answer beyond making the spacings equal: clearing a 14 m survey stack would put the top of the transit stack above 50 m. Vertical separation does not scale.

Timing is modelled optimistically, twice over ACCURACY

The leg-time model ran 16 % optimistic against the 11 Sep flight, and cruise-derived estimates are ~50 % optimistic because of the corner cap. Duration, endurance and "time until the area is searched" all inherit both errors, and the planner presents them without a confidence range.

Lane centring leaves the boundary short COVERAGE

Lanes are centred within each cell, so the outermost swath can stop short of the OpsArea edge. Measured on the SAFTI urban sector: 1.3–14.3 % of boundary samples outside every camera swath, worst 9.5 m. It also flags an existing flown sector, SAFTI City, at 0.9–1.9 %. The planner now measures and reports this, but does not fix it.

The guardrail is timing-agnostic by design CONSERVATISM

Two drones that could never be in the same place at the same time still fail separation if their paths cross at close altitudes, because timing drifts and the check refuses to rely on it. This is the right default and it is also why sequenced or time-separated solutions cannot be expressed at all — the model has no time axis.

A documentation test that checks values, not meaning PROCESS

`parameters.yaml` is asserted against the code, yet it survived two days with 40 duplicated lines spliced into the middle of a sentence. The YAML still parsed and every value it knew about still matched, so all tests passed. Structural integrity of the reference is unchecked.

The gate does not scale as a fairness device DESIGN

Equalising N arrival times with a point that has two degrees of freedom works for three drones and degrades from four onward — 0.2 s at three, 30–34 s at eight. Position alone is the wrong instrument past a small fleet; speed trimming or staggered departure is the mechanism that scales.

08

Capability limits

These are not defects in the code. They are the edges of what this thing is currently able to be asked.

WHAT A MISSION PLANNER MIGHT DO, AND WHETHER THIS ONE DOES

CAPABILITY	STATE	CONSEQUENCE
Disjoint or holed search areas	Not supported	One OpsArea polygon only. A no-fly building must be modelled as a concavity reaching the boundary, not a hole.
Terrain	Absent	Flat-earth equirectangular projection, constant altitude relative to launch. No DEM, so AGL is only nominal over sloping ground.
Wind	Absent	Speeds, durations and battery figures assume still air.
Obstacle avoidance	Absent	Deconfliction is geometric and pre-flight. No sense-and-avoid, no inter-vehicle comms.
In-flight replanning	Absent	Plans are built, uploaded once and flown to the end. A drone dropping out means its cell is simply not searched.
Distribution to aircraft	Manual	One file per drone, uploaded by hand. No addressing, no acknowledgement, no check that the aircraft flying is the one the plan was written for.
Sequenced return	Absent	Every drone flies its own RTL at PX4's default 5 m/s, and the 28 m drone climbs to 30 m to come home — collapsing the altitude layering exactly when they converge.
Launch scheduling	Absent	All drones climb from and descend to one GCS point through each other's layers. The planner warns; it cannot fix.
Endurance feasibility	Cannot answer	See §7. Relative energy only.
Hardware validation	None	Every result is SITL. The physical controller is a MicoAir743-AIO on PX4 v1.16.2 against a v1.16.0 simulation — the versions do not match.

09

What follows

Ranked by what unblocks the most, rather than by effort.

1. **Measure the airframe.** Hover current against mass, cruise power against speed. Until this exists the planner cannot say whether a plan is flyable, which is the most basic question a mission planner should answer.
2. **Fly 3.5 m/s².** It is set and unflown. The 18 Sep result is only half an experiment until the correction is measured.
3. **Close the SITL-to-hardware gap.** Reconcile v1.16.0 against v1.16.2, then repeat one known sortie on the real controller. Every number in §5 is currently a simulation result.
4. **Decide where the decisions live.** Ground station, controller, or companion computer. In-flight replanning, the MAVLink router, and sequenced return are all really the same question about that boundary, and answering it once settles three parked items.
5. **Offboard trajectory streaming.** The only remaining route to a steady bank, and the thing four waypoint drawings failed to achieve.

Sources: `history.md` (entries 01–22 Sep 2026), `FlightPlanner/parameters.yaml`, `sar_turn_params.params` tuning notes, and the module sources under `src/flight_planner/`. Flight figures are from PX4 SITL logs analysed with `pyulog`. Sector outlines for Bedok Reservoir and the SAFTI urban blocks are drawn approximations, not survey data.